SOFTWARE REQUIREMENTS SPECIFICATION

SMART CALENDAR APPLICATIONS

Document Type	Software Requirements Specification (SRS)
Project	Smart Calendar Applications
Version	1.0
Prepared For	Academic / Student Project
Date	09 October 2026

Prepared as a project documentation template. Update the implementation details to match your final application.

Document Overview

This document describes the purpose, scope, functional requirements, interface requirements, quality requirements, data requirements, use cases, and acceptance criteria for Smart Calendar Applications.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the Smart Calendar Applications project. It is intended for students, developers, testers, project guides, and other stakeholders.

1.2 Scope

Smart Calendar Applications is a calendar and task-planning system that helps users create events, organize tasks, set reminders, view schedules, and manage their daily activities from one place. The project can be developed as a web or mobile application. The initial version focuses on personal scheduling and does not require third-party calendar integration.

1.3 Intended Audience

The intended audience includes the project team, college project guide, software developers, testers, and end users.

1.4 Definitions and Abbreviations

SRS: Software Requirements Specification; UI: User Interface; DB: Database; Event: an appointment or scheduled activity; Reminder: a notification sent before an event or task.

2. Overall Description

2.1 Product Perspective

The application is a standalone scheduling system with a user interface, application logic, and database. It stores user accounts, events, tasks, and reminder settings. Notification delivery depends on the platform and permissions available.

2.2 Product Functions

The system allows users to register and sign in, create and edit events, create and track tasks, view a calendar by day/week/month, set reminders, search schedules, and manage profile and preferences.

2.3 User Classes

Guest: can access the sign-in or registration screen. Registered User: can manage personal events, tasks, reminders, and settings. Administrator (optional for the student prototype): can manage basic application configuration and support user account maintenance.

2.4 Operating Environment

Proposed environment: Android mobile device or modern web browser; application server or local application logic; relational or document database; internet connection for online synchronization and remote notifications, if implemented.

2.5 Constraints

The application must respect user privacy, validate date and time inputs, and request notification permissions where required. Cloud sync and external calendar integration are optional extensions, not mandatory features of the initial version.

2.6 Assumptions and Dependencies

Users have access to a supported device and can enter valid dates and times. The system clock and time zone are configured correctly. Reminder notifications depend on operating-system settings and permissions.

3. Functional Requirements

FR-01: User Registration

The system shall allow a new user to create an account using the required registration fields.

FR-02: User Login and Logout

The system shall authenticate registered users and allow them to log out securely.

FR-03: Manage Events

The system shall allow users to add, view, edit, and delete calendar events. Each event shall include a title, date, start time, and optional description, location, and end time.

FR-04: Calendar Views

The system shall display events in day, week, and month views. Users shall be able to navigate to previous and next dates or months.

FR-05: Manage Tasks

The system shall allow users to create, edit, delete, and mark tasks as completed. A task may include a title, due date, priority, and status.

FR-06: Set Reminders

The system shall allow users to set or change a reminder time for an event or task.

FR-07: Reminder Notifications

The system shall notify users about scheduled events or due tasks when notifications are enabled and supported by the platform.

FR-08: Search and Filter

The system shall allow users to search events and tasks by keywords and filter them by date or completion status.

FR-09: Profile and Preferences

The system shall allow users to update supported profile details and preferences, such as default calendar view and notification settings.

FR-10: Data Persistence

The system shall save created or updated events and tasks and display the saved information when the user returns to the application.

FR-11: Input Validation

The system shall validate required fields and date/time values and display a clear message when input is invalid.

FR-12: Conflict Warning

The system should warn the user when a new event overlaps with another event in the same calendar, where overlap detection is implemented.

4. External Interface Requirements

4.1 User Interface

The interface should provide a clear dashboard, calendar navigation, event/task forms, reminder controls, search, and profile/settings screens. Labels and error messages should be simple and understandable.

4.2 Hardware Interface

The system may use a phone or computer display, keyboard/touch input, and device notification capability. No special hardware is required.

4.3 Software Interface

The application may use a database and the operating system's notification service. Optional future integrations may include email, cloud storage, or external calendar APIs.

4.4 Communication Interface

If implemented as an online application, communication between the client and server should use secure HTTPS connections. An offline/local prototype may store data on the device.

5. Non-Functional Requirements

NFR-01: Usability

A first-time user should be able to create an event using a small number of clear steps.

NFR-02: Performance

Common actions such as opening the calendar, viewing events, and saving a record should normally complete within 2 seconds under typical project-demo conditions.

NFR-03: Security

The system shall protect account access, avoid storing plain-text passwords, and restrict each user to their own personal schedule.

NFR-04: Reliability

The system should save data consistently and show an understandable error if a save or notification operation fails.

NFR-05: Availability

For an online deployment, the application should be available whenever its hosting service is operational. A local prototype may work without internet except for online features.

NFR-06: Maintainability

The code should be organized into understandable modules such as authentication, events, tasks, reminders, and data access.

NFR-07: Compatibility

The UI should adapt to commonly used screen sizes and supported modern browsers or the selected mobile platform.

NFR-08: Privacy

Only data needed for scheduling should be collected, and users should be informed about notification permissions and data handling.

6. Data Requirements

6.1 Main Data Entities

User: user_id, name, email/username, password_hash, preferences. Event: event_id, user_id, title, description, date, start_time, end_time, location, reminder_time. Task: task_id, user_id, title, due_date, priority, status, reminder_time. Notification: notification_id, user_id, related_item_id, scheduled_time, delivery_status.

6.2 Data Rules

Every event and task must belong to a user. Event title and date/time are required. End time must not be earlier than start time when both are supplied. Task status must use a defined value such as Pending or Completed. Deleting an item should also remove or deactivate its related reminder.

6.3 Backup and Recovery

If cloud storage is used, the system should support regular backups. For a local prototype, the project documentation should state how data is stored and what happens if the app is removed or device data is cleared.

7. Use Cases

UC-01: Sign In

Actor: Registered User. Precondition: An account exists. Main flow: User enters credentials; system validates them; dashboard is displayed. Alternative flow: If credentials are invalid, the system displays an error and allows retry.

UC-02: Create Event

Actor: Registered User. Precondition: User is signed in. Main flow: User selects Add Event, enters event details, optionally sets a reminder, and saves. System validates the fields and stores the event. Alternative flow: Invalid fields are highlighted and the event is not saved until corrected.

UC-03: Create Task

Actor: Registered User. Precondition: User is signed in. Main flow: User enters task details and due date, selects priority if needed, and saves. The task appears in the task list and relevant calendar view.

UC-04: Complete Task

Actor: Registered User. Precondition: The task exists. Main flow: User marks the task completed; the system updates its status and reflects the change in the task list.

UC-05: Receive Reminder

Actor: Registered User / Notification Service. Precondition: A reminder is set and notifications are permitted. Main flow: At the scheduled time, the platform attempts to display a notification. If notifications are disabled or unavailable, delivery may not occur.

8. System Models (Textual)

8.1 High-Level Architecture

User Interface → Application Logic (Authentication, Calendar, Tasks, Reminders) → Database. The reminder module communicates with the device or platform notification service.

8.2 Basic Workflow

Sign in → Open Dashboard → Select Calendar or Tasks → Add/Edit Item → Validate Input → Save to Database → Schedule Reminder if requested → Display updated schedule.

9. Acceptance Criteria

AC-01

A registered user can sign in and log out.

AC-02

A user can create, view, edit, and delete an event.

AC-03

Day, week, and month calendar views show saved events on the correct dates.

AC-04

A user can create a task and change its status to Completed.

AC-05

A reminder can be configured; notification behavior is verified on the target platform with permission enabled.

AC-06

Invalid or missing required fields produce a clear validation message.

AC-07

A user cannot view or modify another user's private schedule.

10. Future Enhancements

Potential Enhancements

Recurring events; shared calendars; group scheduling; email/SMS reminders; cloud synchronization across devices; AI-based schedule suggestions; integration with Google Calendar or other calendar services; multilingual interface.

11. Conclusion

Summary

This SRS defines the proposed scope and requirements for Smart Calendar Applications. It provides a baseline for design, implementation, testing, and project evaluation. Requirements may be refined with the project guide before development begins.